JavaScript Objects and Arrays

1. Objects in JavaScript

1.1 Concept of Object

In JavaScript, **everything is an object**. Even functions are objects.

An object is a collection of:

- **Properties** (data members / fields)
- **Methods** (functions inside an object)

Key Characteristics

- Objects are **dynamic**
- Properties can be **added, modified, or deleted** at runtime
- Properties can be accessed using:
  - Dot operator (`.`)
  - Subscript operator (`[]`)

---

2. Ways to Create Objects in JavaScript

JavaScript provides multiple ways to create objects:

1. Object Literal `{}`
2. `new Object()`
3. Constructor Function

---

3. Object Creation Using Object Literal {}

This is the **most commonly used and recommended** approach.

```
let i = {
  name: "Pencil",
  company: "Natraj",
  price: 5.0,
};

console.log("type of i = " + typeof i);
console.log("i name = ", i.name);
```

```
console.log("i company = ", i.company);
console.log("i price = ", i["price"]);
```

Advantages

- Simple and readable
- JSON-like syntax
- No need for `new` keyword

---

4. Object Creation Using `new Object()`

Objects can be created using the built-in `Object` constructor.

Using Dot Operator

```
let p = new Object();
p.name = "James Bond";
p.age = 65;
p.addr = "London";

console.log("type of p = " + typeof p);
console.log("p name = ", p.name);
console.log("p age = ", p.age);
console.log("p addr = ", p.addr);
```

Using Subscript Operator `[]`

```
let m = new Object();
m["model"] = "iPhone 14 Pro";
m["company"] = "Apple";
m.price = 89000.0;

console.log("type of m = " + typeof m);
console.log("m model = ", m["model"]);
console.log("m company = ", m.company);
console.log("m price = ", m["price"]);
```

When to Use `[]`

- Property name stored in a variable

- Property name contains spaces or special characters

---

5. Object Creation Using Constructor Function

A constructor function is used to create **multiple objects with similar structure**.

Rules

- Function name starts with a **capital letter** (convention)
- Use `this` keyword to refer to the current object
- Objects are created using `new` keyword

Example

```
function Car(model, company, price) {
  this.model = model;
  this.company = company;
  this.price = price;
}
```

---

6. Prototype in JavaScript

To add **common properties and methods** shared by all objects, JavaScript uses **prototype**.

```
Car.prototype.color = "red";

Car.prototype.printInfo = function () {
  console.log("car model = ", this.model);
  console.log("car company = ", this.company);
  console.log("car price = ", this.price);
  console.log("car color = ", this.color);
};
```

Creating Objects

```
let c1 = new Car("i10", "Hyundai", 800000.0);
c1.printInfo();

let c2 = new Car("Punch", "Tata", 900000.0);
c2.printInfo();
```

Benefits of Prototype

- Memory efficient
- Shared behavior across objects

---

7. Arrays in JavaScript

An array is a **special type of object** that stores multiple values.

Key Points

- Index-based (0 to n-1)
- Can store same or mixed data types
- Arrays have built-in properties and methods

---

8. Array Declaration

8.1 Using Square Brackets (Recommended)

```
let fruits = ["Apple", "Banana", "Mango"];
```

8.2 Using Array Constructor

```
let fruits = new Array("Apple", "Banana", "Mango");
```

---

9. Iterating Arrays

9.1 Using `for` Loop

```
for (let i = 0; i < fruits.length; i++) {
  console.log("fruits[" + i + "] = " + fruits[i]);
}
```

9.2 Using `for...of` Loop

```
for (let f of fruits) {
  console.log("element: ", f);
}
```

---

10. Array of Objects

Arrays can store objects using JSON syntax.

```
let mobiles = [
  { model: "iPhone 14 Pro", company: "Apple", price: 89000.0 },
  { model: "Google Pixel 9", company: "Google", price: 92000.0 },
  { model: "Galaxy S25", company: "Samsung", price: 120000.0 },
];

for (let i = 0; i < mobiles.length; i++) {
  console.log(mobiles[i].model + " | " + mobiles[i].company + " | " +
mobiles[i].price);
}
```

---

11. Array Operations

11.1 `push()` – Add Elements at End

The `push()` method adds one or more elements to the **end of an array** and returns the new length of the array.

```
let names = ["Nilesh", "Rajiv", "Rohan"];
names.push("Nitin");
names.push("Prashant");
```

11.2 `splice()` – Insert Elements/Delete Elements

The `splice()` method can be used to **insert elements at a specific index** without deleting existing elements.

The `splice()` method can also be used to **delete elements** from a specific position.

Syntax:

```
array.splice(startIndex, deleteCount, item1, item2, ...);
```

```
names.splice(1, 0, "Sarang", "Vishal"); //Insert
names.splice(2, 1); //Delete
```

11.3 `pop()` – Remove Last Element

The `pop()` method removes the **last element** from an array and returns the removed element.

```
let n1 = names.pop();
```

11.4 `at()` – Access Elements (Supports Negative Index)

The `at()` method is used to access elements by index and also supports **negative indexing**.

```
console.log(names.at(3));
console.log(names.at(-1)); // last element
```

11.5 `filter()` – Select Elements Based on Condition

The `filter()` method creates a **new array** containing elements that satisfy a given condition. The original array remains unchanged.

```
let names = ["Nilesh", "Rajiv", "Rohan", "Nitin", "Prashant"];

let longNames = names.filter(name => name.length > 5);

console.log(longNames);
// Output: ["Nilesh", "Prashant"]
```

Key Points:

- Returns a new array
- Used for searching and filtering data

- Does not modify the original array

11.6 `map()` – Transform Each Element

The `map()` method creates a **new array** by transforming each element of the original array.

```
let names = ["Nilesh", "Rajiv", "Rohan", "Nitin"];

let upperNames = names.map(name => name.toUpperCase());

console.log(upperNames);
// Output: ["NILESH", "RAJIV", "ROHAN", "NITIN"]
```

Another Example:

```
let formattedNames = names.map(name => "Mr. " + name);
```

Key Points:

- Returns a new array
- Used for data transformation
- Commonly used in UI rendering (React, Angular)

---

12. Array Traversal

Forward Traversal

```
for (let i = 0; i < names.length; i++) {
  console.log("fwd : " + names[i]);
}
```

Reverse Traversal

```
for (let i = 0; i < names.length; i++) {
  console.log("rev : " + names.at(-i - 1));
}
```